Colégio Técnico da UFMG
Disciplina: Programação Orientada a Objetos
Professor: Pedro Montoaneli

Lista 1

1. Modifique a classe `Disciplina` desenvolvida em sala na Aula 4 como descrito a seguir.

(a) Inclua um segundo membro de dados do tipo `string` representando o nome do professor.

(b) Forneça `setNomeProfessor` e `getNomeProfessor`.

(c) Modifique o construtor para receber dois parâmetros, o nome do curso e o nome do professor.

(d) Modifique o método `mensagem()` para que ele primeiro imprima a mensagem de boas-vindas com o nome do curso e depois imprima "Esta disciplina é ministrada por: " seguido do nome do professor.

Demonstre a classe em um programa que cria dois objetos com nomes diferentes e chama `mensagem()` nos dois.

2. Crie uma classe `Account` que um banco poderia usar para representar contas bancárias. Ela deve ter um atributo privado `saldo` do tipo `float`. Forneça um construtor que recebe o saldo inicial e o valida, configurando o saldo como zero e exibindo uma mensagem de erro se o valor for negativo. Forneça três métodos: `creditar(float valor)` que adiciona ao saldo, `debitar(float valor)` que subtrai do saldo apenas se o valor não exceder o saldo atual, imprimindo uma mensagem de erro caso contrário, e `getSaldo()` que retorna o saldo. Crie um programa com dois objetos `Account` e demonstre os três métodos, incluindo tentativas inválidas.

3. Crie uma classe `Sensor` com os atributos privados `nome` (`string`), `temperatura` (`float`) e `unidade` (`char`, valendo `'C'` para Celsius ou `'F'` para Fahrenheit). Forneça dois construtores, um sem parâmetros que inicializa `nome` como "Sensor", `temperatura` como 0 e `unidade` como `'C'`, e um com os três parâmetros. O setter de temperatura deve rejeitar valores abaixo de $-273{,}15$ quando a unidade for Celsius, e abaixo de $-459{,}67$ quando for Fahrenheit, que são os zeros absolutos em cada escala. Forneça um método `leitura()` que imprime o nome do sensor seguido da temperatura e da unidade, por exemplo "Sensor1: 36.5 C". No `main`, crie ao menos três sensores, tente registrar valores inválidos e chame `leitura()` em todos.

4. Crie uma classe `Retangulo` com atributos privados `largura` e `altura`, ambos do tipo `float`. Os dois setters devem rejeitar valores negativos ou zero. Forneça dois construtores, um

sem parâmetros que inicializa ambos como `1.0`, e um com os dois parâmetros. Adicione os métodos `area()` e `perimetro()`, que calculam e retornam os respectivos valores, e um método `exibir()` que imprime todos os dados do retângulo. Por fim, adicione um método `quadrado()` que retorna `true` se o retângulo for um quadrado.

No `main`, crie um array de três retângulos com dimensões diferentes. Percorra o array imprimindo os dados de cada um, e ao final imprima qual deles tem a maior área. O array deve conter ao menos um quadrado.

5. Crie uma classe `Placar` para um jogo entre dois times. Os atributos privados devem ser `nomeTime1`, `nomeTime2` (`string`) e `golsTime1`, `golsTime2` (`int`). O construtor recebe os nomes dos dois times e inicializa os gols como zero. Forneça métodos `marcarTime1()` e `marcarTime2()` que incrementam os gols do respectivo time, e um método `resultado()` que imprime o placar atual no formato "Atletico 2 x 1 Cruzeiro", seguido de quem está vencendo ou se é empate. Adicione um método `reiniciar()` que zera os gols sem alterar os nomes, e um destrutor que imprime "Partida encerrada: " seguido do resultado final.

No `main`, simule uma partida com pelo menos cinco gols, chame `resultado()` após cada gol e `reiniciar()` para uma segunda partida.

6. Crie uma classe `Produto` com os atributos privados `nome` (`string`), `preco` (`float`) e `estoque` (`int`). O construtor recebe os três valores e valida que o preço seja positivo e o estoque seja maior ou igual a zero. Forneça setters com validação para preço e estoque. Adicione três métodos de operação: `vender(int quantidade)` que reduz o estoque se houver unidades suficientes, imprimindo uma mensagem de erro caso contrário, `repor(int quantidade)` que aumenta o estoque apenas para valores positivos, e `valorEmEstoque()` que retorna o preço multiplicado pelo estoque atual.

Crie uma função `void maisValioso(Produto& a, Produto& b)` que recebe dois produtos por referência e imprime qual deles tem o maior valor em estoque. No `main`, crie dois produtos, faça operações de venda e reposição e chame `maisValioso`.

7. Crie uma classe `Aluno` com os atributos privados `nome` (`string`), `matricula` (`int`) e um array privado `notas[4]` (`float`) representando quatro avaliações. O construtor recebe nome e matrícula e inicializa todas as notas como zero. Forneça um método `lancarNota(int indice, float valor)` que registra a nota na posição indicada, validando que o índice esteja entre 0 e 3 e que o valor esteja entre 0 e 10. Adicione `mediaFinal()` que calcula e retorna a média das quatro notas, e `situacao()` que imprime "Aprovado" para média maior ou igual a 6 e "Reprovado" caso contrário.

A entrega deste exercício deve estar obrigatoriamente separada em três arquivos: `Aluno.h`, `Aluno.cpp` e `main.cpp`. O arquivo `Aluno.h` deve conter a guarda de inclusão. No `main`, crie dois alunos, lance todas as notas e exiba a situação de cada um. Inclua no comentário do `main.cpp` o comando de compilação que você usou.

8. Crie uma classe `Reserva` para gerenciar reservas de uma pequena pousada. Os atributos privados devem ser `nomeHospede` (`string`), `numeroQuarto` (`int`, entre 1 e 10), `diarias` (`int`, mínimo 1) e `precoPorDiaria` (`float`). Forneça um construtor que recebe os quatro valores e valida cada um, recusando a construção com mensagem de erro caso qualquer valor seja inválido e usando valores padrão nesse caso, como quarto 1, uma diária e preço zero. Adicione `valorTotal()` que retorna o total a pagar, `extrato()` que imprime todos os dados formatados da reserva, e `prorrogar(int dias)` que acrescenta dias à reserva apenas para valores positivos.

Crie também uma função livre `maisLonga(Reserva& a, Reserva& b)` que imprime qual reserva tem mais diárias.

No `main`, crie um array de três reservas, uma delas com dados inválidos para mostrar o tratamento de erro. Percorra o array chamando `extrato()` em cada uma, exiba o valor total de todas as reservas somado e chame `maisLonga` para comparar as duas primeiras. **Este exercício deve ser entregue separado em arquivos**, com `Reserva.h`, `Reserva.cpp` e `main.cpp`.